Lab 07 — OpenSSL, a local CA, and TLS fields

Course: Maxim Nyansa Cybersecurity Bootcamp, Ghana
Trainee: James Ackah Blay · Aiyinasi, Ghana
Week: 7 — Cryptography and secure communications
Estimated time: 4 hours
Difficulty: Intermediate. You will create keys and throw the private ones away from git.

> **Tutor note.** Work in `$HOME/lab-07-work`, which is outside this repository. Do not commit `*.key`. A self-signed name match is not a trust decision. The Python exercises in the Week 7 folders remain the hashing and Fernet practice. This lab is the command line.

1. Learning objectives

Objective	Job skill
Hash a file and say what a hash is not	Integrity checks on evidence
Create a tiny lab CA and a server certificate	PKI vocabulary: key, CSR, certificate, issuer
Read subject, issuer, dates, and SAN	The questions you ask before trusting a dashboard
Explain TLS and IPsec without a diagram full of magic	Data in transit, in a handover or an interview

2. Prerequisites and setup

OpenSSL on Ubuntu (`openssl version`). The Week 7 scripts:

```
        python3 -m pytest week-07-cryptography/encryption-hashing-practice week-07-
cryptography/certificate-inspection
```

Create the work directory outside the repo:

```
        mkdir -p "$HOME/lab-07-work"
        cd "$HOME/lab-07-work"
```

3. Scenario

The Wazuh dashboard will present a certificate. Before you click through a browser warning, you should be able to say who issued it, which name it claims, and when it expires. You will build a tiny CA so those fields are yours to inspect, then compare that habit with the Python inspector.

4. Tasks

Task 1 — Hash, do not encrypt

```
        echo 'lab evidence' > note.txt
        openssl dgst -sha256 note.txt
```

Change one letter and hash again.

> **You should now see** two different digests. Write one sentence: a hash does not hide the file, and you cannot reverse this digest back into `note.txt`.

Password storage is a different tool. The Python exercise uses scrypt. Do not store a password with a single fast hash and call it done.

Task 2 — Lab CA and a server certificate

```
        openssl genrsa -out ca.key 2048
        openssl req -x509 -new -nodes -key ca.key -sha256 -days 30 -subj "/CN=Lab CA" -out
ca.crt
        openssl genrsa -out server.key 2048
        openssl req -new -key server.key -subj "/CN=lab-wazuh.home" -out server.csr
        openssl x509 -req -in server.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out
server.crt -days 30 -sha256
        openssl x509 -in server.crt -noout -subject -issuer -dates
```

Confirm the private keys are not in the repository:

```
        find "$HOME/lab-07-work" -name '*.key' -printf '%p\n'
        git -C /path/to/CyberSecurity status --short
```

Use your real clone path. `git status` should not list `ca.key` or `server.key`.

Task 3 — Inspect with Python

Copy only `server.crt` if you want the inspector to read it. From `week-07-cryptography/certificate-inspection`:

```
        python3 cert_inspect.py "$HOME/lab-07-work/server.crt"
```

`self_signed_name` should be false this time, because the issuer is `Lab CA` and the subject is `lab-wazuh.home`. That still does not mean your laptop trusts the CA. Trust is a separate choice.

Task 4 — Say TLS and IPsec apart

In `../secure-communications/tls-ipsec-explainer.md`, replace the closing paragraph in your own words. Use the certificate you just read as the mini demo. TLS wraps one application session, such as the dashboard. IPsec wraps IP packets, often as a tunnel, and can cover many applications. Logs on the far side of a tunnel may show the inner addresses. Logs in the middle may show only the tunnel.

5. Troubleshooting

What you see	What it means
`Can't open ca.key`	You are not in `$HOME/lab-07-work`, or Task 2 started halfway
Browser still warns	The lab CA is not in the trust store. That is expected
`self_signed_name` is true	You inspected `ca.crt`, not `server.crt`
A key appears in `git status`	Move it out of the repo and do not commit it. Rotate it if it was pushed

6. Stretch task

Add a subject alternative name. One way is a small config file next to the CSR in the work directory, with `subjectAltName=DNS:lab-wazuh.home`, passed to `openssl x509 -req` with `-extfile`. Re-inspect. If you cannot get the SAN in, write the command that failed and the error. Do not claim the SAN is present unless the inspector prints it.

7. Evidence

- Two SHA-256 lines, before and after the one-letter change.
- `openssl x509 -noout` output for `server.crt`.
- Python inspector JSON.
- A sentence on TLS versus IPsec.
- Confirmation the keys are outside git.

8. Reflection

1. What did the CA sign, the key or the certificate request?
2. Why is a 30-day lab certificate a feature, not an annoyance?
3. What would you check on the Wazuh dashboard certificate before you trusted it?

9. Interview questions

1. What is the difference between hashing and encryption?
2. What does a certificate bind?
3. Why is subject-equals-issuer not the same as “trusted”?
4. When would you choose TLS, and when would a tunnel be the topic instead?

10. Grading rubric

Criterion	Exceeds	Meets	Not yet
Hashing	You explain the one-letter change and what a hash cannot do	Two digests saved	A hash described as encryption
PKI	SAN included, or the failed command is quoted	Subject, issuer, and dates read from `server.crt`	Keys committed
Trust	You say the browser warning is expected	Inspector output saved	“It is secure because OpenSSL ran”
Transit	TLS and IPsec are distinguished in your words	The explainer paragraph is replaced	The template paragraph submitted unchanged